

Quantization

Shrinking a model to fit a laptop · video 2:30

This lesson answers one question: how do people run big models on laptops and phones? A model is mostly weights, normally stored with 16 or 32 bits each. **Quantization** stores each weight with fewer bits: round it to a small grid of integers and keep one scale per row. On a real 0.5B model, **8 bits** is a quarter of the size with practically the same loss; **4 bits** is an eighth, noticeably worse but usable; below that, the model falls apart.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 6).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. Where the size comes from	▶ 0:08–0:35	2	☐
2. Round to a grid, keep a scale	▶ 0:35–1:06	3	☐
3. Measuring the damage	▶ 1:06–1:44	2	☐
4. Subtleties and better methods	▶ 1:44–2:30	2	☐

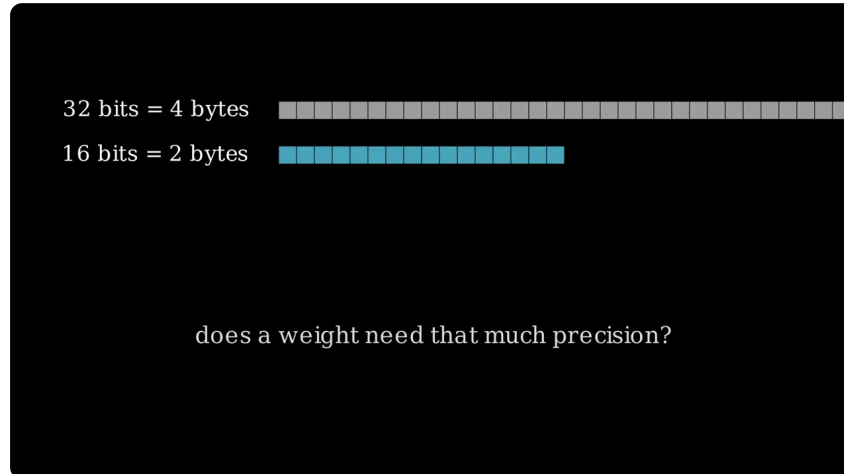
Before you start: Foundations F03 (matrices) and How LLMs Work episode 11 (loss). Code: `code/p10_quantization` (downloads Qwen2.5-0.5B-Instruct, about 1 GB; CPU).

CONCEPT 1 OF 4

Where the size comes from

▶ Video 0:08–0:35

A model is mostly numbers, its **weights**. Qwen2.5-0.5B-Instruct has **494,032,768**. At 32 bits (4 bytes) each, that is about **1.98 GB**; big models need hundreds of GB. Each weight is normally a 32-bit number (4 bytes) or 16 bits (2 bytes). **Quantization** stores each weight with fewer bits.



▶ 0:34 Bits per weight: 32, 16, 8 or 4.

KEY IDEA

Memory $\approx$ number of weights $\times$ bits per weight. Halve the bits, halve the size.

Check 1: Where the size comes from

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

A 7-billion-parameter model is stored in 16-bit. About how many GB is that? And at 4 bits?

1.2 TRUE OR FALSE

“Quantization removes weights from the model.”

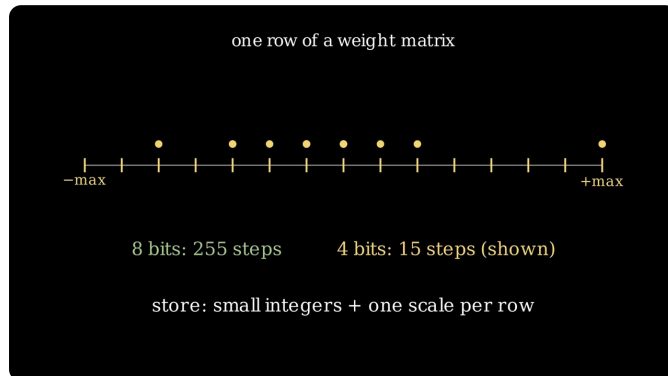
True ☐ False ☐

CONCEPT 2 OF 4

Round to a grid, keep a scale

▶ Video 0:35–1:06

The simplest recipe: take one row of a weight matrix and find its largest absolute value. Round every weight to the nearest step on a small grid from $-\text{max}$ to $+\text{max}$. With **8 bits** that is **255** steps ($-127 \dots 127$); with **4 bits**, only **15** ($-7 \dots 7$). Store the small integers, plus one scale for the row.



▶ 0:54 Round each weight in a row to the nearest step between $-\text{max}$ and $+\text{max}$; store small integers plus one scale.

real weights: layer 1, first row, first six

32-bit	-0.0019	-0.0052	+0.0188	+0.0125	+0.0040	-0.0092
4-bit	0.0000	-0.0082	+0.0165	+0.0165	0.0000	-0.0082

▶ 1:04 Real weights from the first layer, before and after 4-bit rounding: two became exactly zero.

The real first weights of the first layer, before and after 4-bit rounding: $-0.0019 \rightarrow 0$, $-0.0052 \rightarrow -0.0082$, $0.0188 \rightarrow 0.0165 \dots$ Close, but not the same; two small ones became exactly **zero**.

KEY IDEA

weight $\approx$ integer $\times$ scale. Fewer bits = a coarser grid = bigger rounding errors.

Check 2: Round to a grid, keep a scale

Answer all 3 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

A row's largest absolute weight is 0.0329. With 4 bits (integers $-7 \dots 7$), what is the scale, and what does 0.0188 become?

2.2 CALCULATION

How many integer levels does symmetric 3-bit quantization use, and why so few?

2.3 SHORT ANSWER

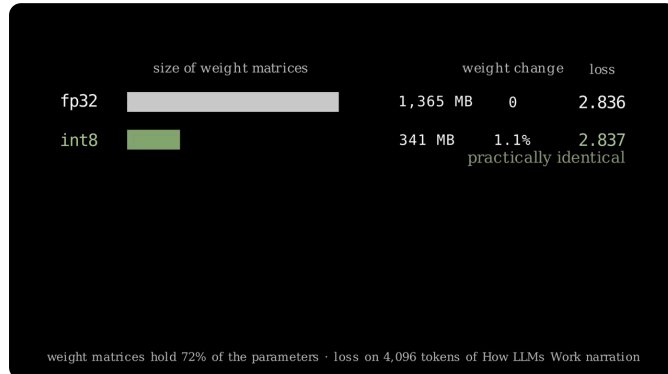
Why does a weight smaller than half a step become exactly zero?

CONCEPT 3 OF 4

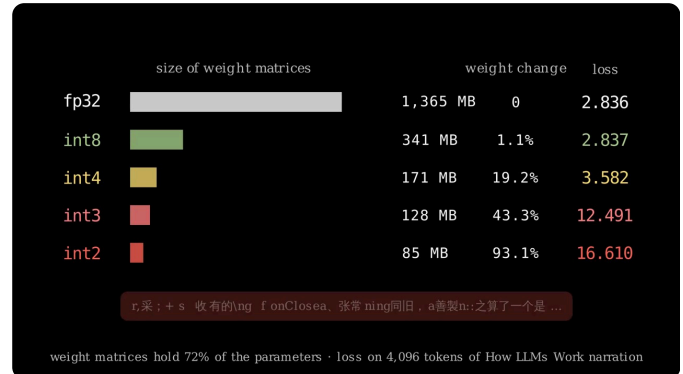
Measuring the damage

▶ Video 1:06–1:44

The weight matrices hold **72%** of the parameters (the rest are mostly the embeddings). Loss on 4,096 tokens of real text (the How LLMs Work narration):



▶ 1:26 fp32 vs int8: a quarter of the size, practically identical loss.



▶ 1:43 Lower bits: int4 is usable; int3 and int2 fall apart.

fp32: 1,365 MB, loss 2.836. **int8:** 341 MB (a quarter), weights move about 1.1%, loss **2.837**: practically identical. **int4:** 171 MB (an eighth), weights move 19%, loss rises to **3.58**; answers still make sense but are vaguer. **int3:** loss **12.5**; **int2:** **16.6**, and the model writes gibberish.

KEY IDEA

Quality holds up well to about 8 bits, degrades at 4 with simple rounding, and collapses below that.

Check 3: Measuring the damage

Answer all 2 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

How many times smaller are the int4 matrices than the fp32 ones?

.....

3.2 PUT IN ORDER

Order from lowest to highest loss: *int2* · *fp32* · *int4* · *int8* · *int3*.

.....

CONCEPT 4 OF 4

Subtleties and better methods

▶ Video 1:44-2:30

At 8 bits the loss barely moved, but the model's answer to our question still **changed**. When two tokens are almost tied, a tiny change can flip the choice, and everything after it (episode 3). Average quality and a single answer are different things to measure.

"In one sentence, what is a context window?"

fp32 A context window refers to the portion of a document or message that contains information about the current conversation or interaction between two parties.

int8 A context window refers to the portion of a video or audio clip that remains visible after the main content has been cut off.

near-tied tokens: a tiny change flips the choice, and everything after it
(neither answer is great: it's a 0.5B model)

▶ 1:56 At int8 the loss barely moved, but the greedy answer changed: near-tied tokens flip.

real tools do better than simple rounding

- ✓ small groups of weights, each with its own scale
- ✓ sensitive weights kept at higher precision
- ✓ calibration on sample text

4-bit versions of big models: a popular trade-off for laptops

▶ 2:13 Real tools: per-group scales, sensitive weights kept precise, calibration.

Real tools do much better than simple rounding: they use **small groups** of weights with their own scale, keep the most **sensitive** weights at higher precision, and **calibrate** on sample text. That is why 4-bit versions of big models are popular. In code, the simple version is three lines per matrix: compute the scale from the largest value, round to integers, multiply back.

KEY IDEA

Quantize, then measure on your own tasks: an unchanged average loss does not guarantee unchanged answers.

Check 4: Subtleties and better methods

Answer all 2 questions before looking at the answers. Pass mark: all correct.

4.1 SHORT ANSWER

Why does a separate scale for each small group of weights help, compared with one scale per row?

4.2 HANDS-ON CODE

Try it yourself. In `code/p10_quantization/quantize.py`, change the loop to `for bits in (6, 5):` and run it. What sizes and losses do you get, and where between 8 and 4 bits does quality start to drop?

```
for bits in (6, 5):
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. Where the size comes from · rewatch ▶ 0:08-0:35

1.1 About 14 GB; about 3.5 GB. $7 \times 10^9 \times 2 \text{ bytes} = 14 \times 10^9 \text{ bytes}$; $\times 0.5 \text{ byte} = 3.5 \times 10^9 \text{ bytes}$ (plus a little for the scales).

1.2 False. Every weight is kept; each one is stored with fewer bits.

2. Round to a grid, keep a scale · rewatch ▶ 0:35-1:06

2.1 Scale 0.0047; 0.0188 → $4 \times 0.0047 = 0.0188$. $0.0329 / 7 = 0.0047$; $0.0188 / 0.0047 = 4.0$, already on the grid. (In the real row, the max is different, so 0.0188 became 0.0165.)

2.2 7 (−3 ... 3). $2^{3-1} - 1 = 3$, so the integers run from −3 to 3: every weight in a row has only 7 possible values.

2.3 Rounding to the nearest grid point sends it to 0, the closest level. With 4 bits, any weight smaller than half the scale rounds to 0.

3. Measuring the damage · rewatch ▶ 1:06-1:44

3.1 8 times. $32 / 4 = 8$; $1,365 \text{ MB} / 8 \approx 171 \text{ MB}$.

3.2 fp32 (2.836) → int8 (2.837) → int4 (3.582) → int3 (12.491) → int2 (16.610). Fewer bits, larger rounding errors, higher loss.

4. Subtleties and better methods · rewatch ▶ 1:44-2:30

4.1 One large weight no longer stretches the grid for the whole row; each group gets a finer grid fitted to its own values. The scale is set by the largest value; smaller groups have smaller maxima, so smaller steps.

4.2 int6: 256 MB, weights move 4.4%, loss 2.875. int5: 213 MB, 9.0%, loss 2.944. Quality starts to drop around 6 bits and falls faster from 5 to 4 bits (3.582). Each bit removed halves the number of grid steps, so the average weight change roughly doubles: $1.1\% \rightarrow 4.4\% \rightarrow 9.0\% \rightarrow 19.2\%$.